



SECURITY ASSESSMENT



Accretion

Provided by Accretion Labs Pte Ltd. for Artemis Capital LLC
September 11, 2026
A26ART1

Artemis Capital LLC

AUDITORS

Role	Name
Auditor	Niklas Brymko (niklas@accretion.xyz)
Lead Auditor	Robert Reith (robert@accretion.xyz)

CLIENT

Artemis Capital LLC (<https://bitcoinhard.money>) engaged Accretion to conduct a security assessment of TBB Staking, a capital management platform providing investment and asset management services.

ENGAGEMENT TIMELINE



AUDITED CODE

#	Program ID	Repository
1	4KgvDmEjPJNtbiVhnZ9Cf1i1vgeZdKrcNKHHVTNTkLWT	https://github.com/jwilliams-74w/tbb-staking

ASSESSMENT

The security assessment of Artemis Capital LLC's TBB Staking program revealed 10 findings: 1 high, 6 low, and 3 informational. The high-severity issue posed a material risk to user funds by enabling permanent denial of unstaking. All 10 findings were remediated by the client prior to report publication, with zero accepted or unresolved risks remaining.

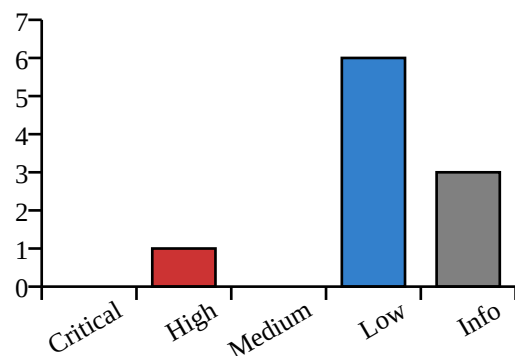
CODE ASSESSMENT

The codebase is a Solana program built with the Anchor framework handling SPL Token and Token-2022 mints. Token extension edge cases and PDA derivation logic showed insufficient validation, though the overall architecture was coherent and well-structured.

KEY FINDINGS

The sole high-severity issue allowed any actor to dust a stake vault with a small token transfer, permanently blocking user unstaking. This represented a grieving vector with direct, irreversible impact on staker fund accessibility. It has been fixed.

SEVERITY DISTRIBUTION



ENGAGEMENT SCOPE

The scope of this security assessment was a full review of the following items:

Item 1: Tbb Staking

Link: <https://github.com/jwilliams-74w/tbb-staking>

Commit: 152740e5b8c05fbea25ff9a995b9254db10cc8f1

Program ID: 4KgvDmEjPJNtbiVhnZ9Cf1i1vgeZdKrCNKhHVTNTkLWT

Audit Result:

- **Audited Commit:** d4b52171e2d6a4a4cb13b5ba0ed9614bd1a48e79

ISSUES SUMMARY

ID	TITLE	SEVERITY	STATUS
ACC-H1	Anyone can dust a stake vault and permanently block unstaking	HIGH	FIXED
ACC-L1	Stake reservations can consume the treasury's permanent one-unit reserve	LOW	FIXED
ACC-L2	Shared global stake nonce enables repeatable staking grieving	LOW	FIXED
ACC-L3	Transfer-fee mints record more principal than the vault receives	LOW	FIXED
ACC-L4	Mutable transfer hooks freeze all live stakes because CPI accounts are discarded	LOW	FIXED
ACC-L5	A Token-2022 permanent delegate bypasses the staking PDAs and can seize principal	LOW	FIXED
ACC-L6	Unchecked mint freeze authority can permanently lock PDA-held stake principal	LOW	FIXED
ACC-I1	Hardcoded Interest table requires whole Program upgrade to change.	INFO	FIXED
ACC-I2	First caller can seize the sole pool authority and brick the intended deployment	INFO	FIXED
ACC-I3	Missing minimum stake validation creates zero-yield locked positions	INFO	FIXED

DETAILED ISSUES

ACC-H1 Anyone can dust a stake vault and permanently block unstaking

HIGH

FIXED

Summary

unstake transfers only the principal recorded in **StakeAccount** and then unconditionally closes the vault, even though the vault can receive unsolicited token transfers. Any holder of the pool mint can send one base unit to a victim's publicly derivable vault; the resulting nonzero remainder makes **CloseAccount** fail and atomically rolls back every withdrawal attempt, while no instruction can sweep the excess.

Impact

Permanent principal freeze; any unprivileged token holder can dust a chosen stake vault without the victim's signature, after which the victim's principal and promised interest remain inaccessible absent a program upgrade.

Location

- [program/programs/tbb-staking/src/lib.rs:170-203](#) (**unstake**)

Code

```
// program/programs/tbb-staking/src/lib.rs:170-203
170 | // 1) Return principal from the stake vault (authority = stake PDA).
171 | transfer_tokens(
172 |     &ctx.accounts.token_program,
173 |     &ctx.accounts.vault,
174 |     &ctx.accounts.mint,
175 |     &ctx.accounts.staker_ata,
176 |     &ctx.accounts.stake_account.to_account_info(),
177 |     stake_acc.amount, // @audit Transfers the recorded amount, not the vault's actual balance.
178 |     ctx.accounts.mint.decimals,
179 |     Some(&[stake_seeds]),
180 | )?;
181 |
182 | // 2) Pay interest from treasury (authority = pool PDA).
183 | transfer_tokens(
184 |     &ctx.accounts.token_program,
185 |     &ctx.accounts.treasury,
186 |     &ctx.accounts.mint,
187 |     &ctx.accounts.staker_ata,
188 |     &ctx.accounts.pool.to_account_info(),
189 |     stake_acc.interest,
190 |     ctx.accounts.mint.decimals,
191 |     Some(&[pool_seeds]),
192 | )?;
193 |
194 | // 3) Close the vault token account, rent to staker.
195 | token_interface::close_account(CpiContext::new_with_signer(
196 |     ctx.accounts.token_program.to_account_info(),
197 |     token_interface::CloseAccount {
198 |         account: ctx.accounts.vault.to_account_info(),
199 |         destination: ctx.accounts.staker.to_account_info(),
200 |         authority: ctx.accounts.stake_account.to_account_info(),
201 |     },
202 |     &[stake_seeds],
203 | ))?; // @audit CloseAccount rejects any nonzero donated remainder.
```

Mitigation Suggestion

In step 1, transfer the full token account balance to the staker, instead of the recorded balance.

Remediation

Fixed in commit **50524af69cb7627d9598596b6ed1d84a596a2389**.

Description

Any token holder can create an otherwise valid position whose promised interest consumes all unreserved treasury funds, including the base unit that the program says must remain forever. Once the position matures, settlement transfers that final unit to the staker and leaves both the treasury balance and `total_promised_interest` at zero. New stakes with nonzero interest then fail the funding check until the authority deposits more tokens.

The attacker receives only their own principal and the interest the program accepted; this path does not expose other users' principal. The practical loss is one base unit plus a temporary availability and defense-in-depth failure, which supports Low severity. No live initialized pool or exploitable production balance was evidenced, so current deployment exploitability remains uncertain.

Root cause

At `programs/tbb-staking/src/lib.rs:216-223`, `stake` computes `available = treasury.amount - pool.total_promised_interest` with checked subtraction but accepts `available >= interest`. Equality is therefore valid, and the newly recorded liability can cover the entire treasury.

This policy conflicts with `withdraw_surplus` at `programs/tbb-staking/src/lib.rs:129-139`, which applies `.saturating_sub(1)` before allowing an administrative withdrawal specifically to keep one base unit in circulation. At `programs/tbb-staking/src/lib.rs:296-324`, `unstake` later pays the recorded interest without a residual-balance check and reduces `total_promised_interest` by the same amount. Settlement preserves the preexisting `treasury.amount - total_promised_interest` gap, so a zero gap admitted by `stake` necessarily permits a zero treasury after all corresponding liabilities settle.

Attack path

1. Begin from a valid pool state with a one-unit treasury and no promised interest. `initialize_pool(initial_funding = 1)` can create this state through the authorized setup path. 2. An unprivileged holder calls public `stake` using the shipped tier 4 (`lock_seconds = 120`, `apr_bps = 1,800`) and a received principal of 1,460,000 base units. 3. `compute_interest` returns `floor(1,460,000 * 1,800 / 10,000 / 31,536,000) = 1`. 4. The check accepts because `available = 1 - 0 = 1` and `available >= interest`. The treasury remains 1 while `total_promised_interest` becomes 1. 5. After 120 seconds, the staker calls `unstake`. The program returns the vault principal, pays one treasury unit as interest, and reduces promised interest to zero. The final state is `treasury.amount = 0` and `total_promised_interest = 0`.

More generally, exploitation requires one or more stake amounts whose computed interest exactly consumes the observable free treasury balance, plus waiting for maturity. Exact attainability depends on the configured tiers, token units, and current balance; the shipped demo tier provides the concrete one-unit case above.

Patch recommendation

Patch recommendation

Finding: `programs/tbb-staking/src/lib.rs:216-223` permits a stake when the unreserved treasury balance exactly equals the newly promised interest. This is inconsistent with `withdraw_surplus`, which preserves one base unit, and allows a later `unstake` to reduce the treasury to zero.

Smallest safe code change

Keep the existing checked subtraction and require the post-reservation remainder to be at least one unit:

```

let available = ctx
  .accounts
  .treasury
  .amount
  .checked_sub(pool.total_promised_interest)
  .ok_or(StakingError::TreasuryUnderfunded)?;
let remaining = available
  .checked_sub(interest)
  .ok_or(StakingError::TreasuryUnderfunded)?;
require!(remaining >= 1, StakingError::TreasuryUnderfunded);

```

This changes only admission of under-reserved stakes. It retains checked arithmetic, avoids an overflow-prone `interest + 1` expression, preserves the existing error behavior, and leaves settlement unchanged. The equivalent `require!(available > interest, ...)` is also safe, but the explicit checked remainder documents and enforces the permanent-unit invariant directly.

Regression-test outline

Add a focused unit/helper test for the reservation rule, or an integration test around `stake` if the project's test harness is expanded:

1. Set `treasury.amount = total_promised_interest + interest` and assert the stake fails with `StakingError::TreasuryUnderfunded`; verify no stake liability is committed.
2. Set `treasury.amount = total_promised_interest + interest + 1` and assert the stake succeeds; verify `pool.total_promised_interest` increases by exactly `interest`.
3. Advance time through the selected lock period and call `unstake`; verify the treasury remains at least one base unit above the remaining promised-interest liability (in the single-position case, exactly one unit), while the staker receives principal plus the recorded interest.
4. Include arithmetic boundary cases: `available = 0`, `interest = 1`, and a case where `available.checked_sub(interest)` underflows; all must return `TreasuryUnderfunded` without state mutation.

Validation: baseline `cargo check` succeeds; existing Anchor `unexpected_cfgs` warnings are unrelated.

Remediation

Fixed in `5f8129e26ad4e1fa77843f73a90cc48ed3c5637d`.

Summary

Every stake account address is derived from the mutable global `pool.total_stakes`, which is incremented after each successful stake. One attacker stake therefore makes all competing transactions signed against the preceding counter fail seed validation; minimum-size stakes and the reachable 120-second tier let the attacker repeatedly advance the counter and recycle its capital.

Impact

Shared-nonce staking denial; any unprivileged token holder can advance the global counter with a minimum-size stake and invalidate already-signed competing stakes, and although each victim can rebuild its transaction, the actor can repeat the interference and reclaim its locked token and account rent after 120 seconds.

Location

- [program/programs/tbb-staking/src/lib.rs:9-15](#) (TIERS)
- [program/programs/tbb-staking/src/lib.rs:81-89](#) (stake)
- [program/programs/tbb-staking/src/lib.rs:125-139](#) (stake)
- [program/programs/tbb-staking/src/lib.rs:219-226](#) (compute_interest)
- [program/programs/tbb-staking/src/lib.rs:351-357](#) (Stake)

Code

```
// program/programs/tbb-staking/src/lib.rs:9-15
9 | pub const TIERS: [(i64, u16); 5] = [
10 |     (30 * 86_400, 500),    // 1 month -> 5% APR
11 |     (90 * 86_400, 800),    // 3 months -> 8% APR
12 |     (180 * 86_400, 1200),  // 6 months -> 12% APR
13 |     (365 * 86_400, 1800),  // 12 months -> 18% APR
14 |     (120, 1800),          // DEMO: 2-minute lock (remove before mainnet)
15 | ];                          // @audit Makes the attacker's stake/rent recyclable after 120 seconds.

// program/programs/tbb-staking/src/lib.rs:81-89
81 | pub fn stake(ctx: Context<Stake>, amount: u64, tier: u8) -> Result<()> {
82 |     require!(amount > 0, StakingError::ZeroAmount); // @audit One base unit is accepted.
83 |     require!((tier as usize) < TIERS.len(), StakingError::InvalidTier);
84 |
85 |     let (lock_seconds, apr_bps) = TIERS[tier as usize];
86 |     let now = Clock::get()?.unix_timestamp;
87 |
88 |     // Interest owed at maturity, computed up front (fixed-term product).
89 |     let interest = compute_interest(amount, apr_bps, lock_seconds)?;

// program/programs/tbb-staking/src/lib.rs:125-139
125 |     stake_acc.stake_index = pool.total_stakes;
126 |     stake_acc.bump = ctx.bumps.stake_account;
127 |
128 |     pool.total_staked = pool
129 |         .total_staked
130 |         .checked_add(amount)
131 |         .ok_or(StakingError::MathOverflow)?;
132 |     pool.total_promised_interest = pool
133 |         .total_promised_interest
134 |         .checked_add(interest)
135 |         .ok_or(StakingError::MathOverflow)?;
136 |     pool.total_stakes = pool
```

```

137 |         .total_stakes
138 |         .checked_add(1)
139 |         .ok_or(StakingError::MathOverflow)?; // @audit Invalidates every transaction built with the
    |         old value.

// program/programs/tbb-staking/src/lib.rs:219-226
219 | fn compute_interest(amount: u64, apr_bps: u16, lock_seconds: i64) -> Result<u64> {
220 |     let interest = (amount as u128)
221 |         .checked_mul(apr_bps as u128)
222 |         .and_then(|v| v.checked_mul(lock_seconds as u128))
223 |         .and_then(|v| v.checked_div(10_000))
224 |         .and_then(|v| v.checked_div(SECONDS_PER_YEAR))
225 |         .ok_or(StakingError::MathOverflow)?;
226 |     u64::try_from(interest).map_err(|_| StakingError::MathOverflow.into())
    |     // @audit Minimum-size stakes round to zero interest and need no treasury reservation.
227 | }

// program/programs/tbb-staking/src/lib.rs:351-357
351 |     #[account(
352 |         init,
353 |         payer = staker,
354 |         space = 8 + StakeAccount::INIT_SPACE,
355 |         seeds = [b"stake", pool.key().as_ref(), staker.key().as_ref(), &pool.total_stakes.to_le_bytes()
    |     ],
    |         // @audit The required address depends on shared mutable state read during validation.
356 |         bump
357 |     )]

```

Remediation

Fixed in [50524af69cb7627d9598596b6ed1d84a596a2389](#).

Summary

`stake` records the requested gross transfer amount without reloading the vault or measuring its actual balance increase. When the configured Token-2022 mint charges a transfer fee, the vault receives less than `stake_acc.amount`; the later withdrawal requests the unavailable gross amount and has no staking-program recovery path.

Impact

Transfer-fee principal shortfall; every normal staker is affected when the legitimately configured Token-2022 mint charges a transfer fee, and that trusted-role/mint-configuration prerequisite caps the otherwise permanent principal freeze at Medium.

Location

- [program/programs/tbb-staking/src/lib.rs:102-124](#) (`stake`)
- [program/programs/tbb-staking/src/lib.rs:170-180](#) (`unstake`)
- [program/programs/tbb-staking/src/lib.rs:229-255](#) (`transfer_tokens`)

Code

```
// program/programs/tbb-staking/src/lib.rs:102-124
102 | // Move principal into the stake vault.
103 | transfer_tokens(
104 |     &ctx.accounts.token_program,
105 |     &ctx.accounts.staker_ata,
106 |     &ctx.accounts.mint,
107 |     &ctx.accounts.vault,
108 |     &ctx.accounts.staker.to_account_info(),
109 |     amount,
110 |     ctx.accounts.mint.decimals,
111 |     None,
112 | )?;
113 |
114 | let stake_acc = &mut ctx.accounts.stake_account;
115 | stake_acc.staker = ctx.accounts.staker.key();
116 | stake_acc.pool = pool.key();
117 | stake_acc.amount = amount; // @audit Stores the gross request without checking the net amount c
redited.
118 | stake_acc.tier = tier;
119 | stake_acc.apr_bps = apr_bps;
120 | stake_acc.start_ts = now;
121 | stake_acc.unlock_ts = now
122 |     .checked_add(lock_seconds)
123 |     .ok_or(StakingError::MathOverflow)?;
124 | stake_acc.interest = interest;

// program/programs/tbb-staking/src/lib.rs:170-180
170 | // 1) Return principal from the stake vault (authority = stake PDA).
171 | transfer_tokens(
172 |     &ctx.accounts.token_program,
173 |     &ctx.accounts.vault,
174 |     &ctx.accounts.mint,
175 |     &ctx.accounts.staker_ata,
176 |     &ctx.accounts.stake_account.to_account_info(),
177 |     stake_acc.amount, // @audit Exceeds the vault balance after an incoming transfer fee.
178 |     ctx.accounts.mint.decimals,
```

```

179 |         Some(&[stake_seeds]),
180 |     )?;

// program/programs/tbb-staking/src/lib.rs:229-255
229 | #[allow(clippy::too_many_arguments)]
230 | fn transfer_tokens<'info>(
231 |     token_program: &Interface<'info, TokenInterface>,
232 |     from: &InterfaceAccount<'info, TokenAccount>,
233 |     mint: &InterfaceAccount<'info, Mint>,
234 |     to: &InterfaceAccount<'info, TokenAccount>,
235 |     authority: &AccountInfo<'info>,
236 |     amount: u64,
237 |     decimals: u8,
238 |     signer_seeds: Option<&[&[u8]]>,
239 | ) -> Result<()> {
240 |     let accounts = TransferChecked {
241 |         from: from.to_account_info(),
242 |         mint: mint.to_account_info(),
243 |         to: to.to_account_info(),
244 |         authority: authority.clone(),
245 |     };
246 |     let cpi = match signer_seeds {
247 |         Some(seeds) => CpiContext::new_with_signer(
248 |             token_program.to_account_info(),
249 |             accounts,
250 |             seeds,
251 |         ),
252 |         None => CpiContext::new(token_program.to_account_info(), accounts),
253 |     };
254 |     token_interface::transfer_checked(cpi, amount, decimals)
255 | } // @audit No pre/post balance delta is measured for Token-2022 transfer semantics.

```

Mitigation Suggestion

Observe the vault balance before and after the transfer and observe the actual transferred amount. Account reload is necessary after the transfer CPI. Then calculate the interest and check for enough balance in the Treasury.

Remediation

Fixed in commit [50524af69cb7627d9598596b6ed1d84a596a2389](#).

Summary

`InitializePool` accepts any Token-2022 mint, including one with a retained `TransferHook` authority, but every token movement is routed through `transfer_tokens`, which accepts and constructs only the four base transfer accounts; the Anchor wrapper consequently cannot forward hook accounts, whereas Token-2022's hook-aware transfer helper explicitly resolves those accounts from an additional-account list.

A reachable sequence is: the pool authority initializes the pool with a hook-capable mint whose hook `program_id` is unset, funds the treasury, and users stake normally; the legitimate hook authority later enables a standard hook, after which every mature user-signed `unstake` fails when Token-2022 attempts to invoke that hook from the empty remaining-account slice, and atomic rollback leaves the stake account, vault, principal, and yield unchanged on every retry until the hook is disabled/reconfigured or the staking program is upgraded.

Impact

All existing positions in the affected pool lose access to principal and promised yield across retries; triggering requires the legitimate pool authority to select a mint carrying a mutable `TransferHook` extension and that mint's legitimate hook authority to activate it, so the trusted-role prerequisite caps the otherwise High temporary-freeze impact at Medium, while mints without that extension are unaffected.

Location

- [program/programs/tbb-staking/src/lib.rs:286-310](#) (`InitializePool`)
- [program/programs/tbb-staking/src/lib.rs:153-203](#) (`unstake`)
- [program/programs/tbb-staking/src/lib.rs:229-255](#) (`transfer_tokens`)

Code

```
// program/programs/tbb-staking/src/lib.rs:286-310
286 #[derive(Accounts)]
287 pub struct InitializePool<'info> {
288     #[account(mut)]
289     pub authority: Signer<'info>,
290     pub mint: InterfaceAccount<'info, Mint>,
291     #[account(
292         init,
293         payer = authority,
294         space = 8 + Pool::INIT_SPACE,
295         seeds = [b"pool"],
296         bump
297     )]
298     pub pool: Account<'info, Pool>,
299     #[account(
300         init,
301         payer = authority,
302         token::mint = mint,
303         token::authority = pool,
304         seeds = [b"treasury"],
305         bump
306     )]
307     pub treasury: InterfaceAccount<'info, TokenAccount>,
308     pub token_program: Interface<'info, TokenInterface>,
309     pub system_program: Program<'info, System>,
310 }

// BUG: The mint is accepted without rejecting or otherwise handling a
```

```

// mutable TransferHook extension.

// program/programs/tbb-staking/src/lib.rs:153-203
153 pub fn unstake(ctx: Context<Unstake>) -> Result<()> {
154     let now = Clock::get()?.unix_timestamp;
155     let stake_acc = &ctx.accounts.stake_account;
156     require!(now >= stake_acc.unlock_ts, StakingError::StillLocked);
157
158     let pool_key = ctx.accounts.pool.key();
159     let staker_key = ctx.accounts.staker.key();
160     let index_bytes = stake_acc.stake_index.to_le_bytes();
161     let stake_seeds: [&[u8]] = &[
162         b"stake",
163         pool_key.as_ref(),
164         staker_key.as_ref(),
165         &index_bytes,
166         &[stake_acc.bump],
167     ];
168     let pool_seeds: [&[u8]] = &[b"pool", &[ctx.accounts.pool.bump]];
169
170     // 1) Return principal from the stake vault (authority = stake PDA).
171     transfer_tokens(
172         &ctx.accounts.token_program,
173         &ctx.accounts.vault,
174         &ctx.accounts.mint,
175         &ctx.accounts.staker_ata,
176         &ctx.accounts.stake_account.to_account_info(),
177         stake_acc.amount,
178         ctx.accounts.mint.decimals,
179         Some(&[stake_seeds]),
180     )?;
181
182     // 2) Pay interest from treasury (authority = pool PDA).
183     transfer_tokens(
184         &ctx.accounts.token_program,
185         &ctx.accounts.treasury,
186         &ctx.accounts.mint,
187         &ctx.accounts.staker_ata,
188         &ctx.accounts.pool.to_account_info(),
189         stake_acc.interest,
190         ctx.accounts.mint.decimals,
191         Some(&[pool_seeds]),
192     )?;
193
194     // 3) Close the vault token account, rent to staker.
195     token_interface::close_account(CpiContext::new_with_signer(
196         ctx.accounts.token_program.to_account_info(),
197         token_interface::CloseAccount {
198             account: ctx.accounts.vault.to_account_info(),
199             destination: ctx.accounts.staker.to_account_info(),
200             authority: ctx.accounts.stake_account.to_account_info(),
201         },
202         &[stake_seeds],
203     ))?;

// BUG: Both payout transfers depend on transfer_tokens. Once a hook is active,
// the principal CPI fails before payout, accounting updates, or account closure.

// program/programs/tbb-staking/src/lib.rs:229-255
229 #[allow(clippy::too_many_arguments)]
230 fn transfer_tokens<'info>(
231     token_program: &Interface<'info, TokenInterface>,
232     from: &InterfaceAccount<'info, TokenAccount>,
233     mint: &InterfaceAccount<'info, Mint>,
234     to: &InterfaceAccount<'info, TokenAccount>,
235     authority: &AccountInfo<'info>,
236     amount: u64,
237     decimals: u8,
238     signer_seeds: Option<&[&[u8]]>,
239 ) -> Result<()> {
240     let accounts = TransferChecked {
241         from: from.to_account_info(),

```

```

242     mint: mint.to_account_info(),
243     to: to.to_account_info(),
244     authority: authority.clone(),
245 };
246 let cpi = match signer_seeds {
247     Some(seeds) => CpiContext::new_with_signer(
248         token_program.to_account_info(),
249         accounts,
250         seeds,
251     ),
252     None => CpiContext::new(token_program.to_account_info(), accounts),
253 };
254 token_interface::transfer_checked(cpi, amount, decimals)
255 }

// BUG: There is no additional/remaining-account parameter, and neither CPI
// context appends the hook program, validation account, or hook-specific accounts.

```

Mitigation Suggestion

During pool initialization, reject Token-2022 mints with a **TransferHook** extension unless every token CPI resolves and forwards that hook's required additional accounts. This prevents a later hook activation from making existing payouts unexecutable.

Further, the initialization should ensure that at least 1 unit is always in circulation. This prevents an initializer from passing the init check, then closing the mint and reopening it with the **TransferHook**. This can be done by ensuring the treasury is funded in the same IX and that it always has at least 1 unit of funding.

Remediation

Fixed in commit [50524af69cb7627d9598596b6ed1d84a596a2389](#).

Summary

`initialize_pool` permanently accepts any `TokenInterface` mint without rejecting a non-null Token-2022 `PermanentDelegate`, while the treasury and per-stake vaults rely only on their ordinary token-account owner being the pool or stake PDA. Token-2022 permits initialization of these accounts with only a seizure warning and accepts the permanent delegate instead of the token-account owner for transfers and burns, so the configured PDA is not the exclusive custodian.

Impact

Principal theft; the configured mint's legitimate permanent-delegate signer can remove stake-vault balances without either the staker or the protocol PDA, but that trusted-role prerequisite caps the otherwise Critical theft impact at Medium.

Severity calibration

The reachable sequence starts with the intended pool authority selecting a valid Token-2022 mint that already has a live permanent delegate; this is a standard supported mint configuration and does not require an attacker to hijack initialization. A user then stakes normally, and the principal is credited to a newly created vault whose ordinary owner is the `stake_account` PDA.

At any time afterward, the current permanent-delegate signer can submit a direct Token-2022 `TransferChecked` or `BurnChecked` instruction against that vault. No staker, pool-authority, pool-PDA, or stake-PDA signature is required. The delegate's transaction commits independently and leaves the on-chain `StakeAccount.amount` unchanged. A later `unstake` therefore attempts to transfer the original recorded amount from an underfunded vault and fails; atomic rollback of that failed unstake cannot undo the earlier delegate transfer or burn.

The affected party is the owner of each targeted stake. There is no reconciliation, principal-only exit, or emergency closure instruction; recovery requires the delegate to return the assets or another party to replace the missing principal. The impact is bounded to the configured mint balance held by targeted stake vaults, with the same authority also capable of draining the reward treasury. Because exploitation requires an existing legitimate mint role and does not grant or expand that role for an unprivileged actor, the project's trusted-role cap applies.

Location

- [program/programs/tbb-staking/src/lib.rs:21-30](#) (`initialize_pool`)
- [program/programs/tbb-staking/src/lib.rs:286-310](#) (`InitializePool`)
- [program/programs/tbb-staking/src/lib.rs:359-367](#) (`Stake`)
- [program/programs/tbb-staking/src/lib.rs:170-180](#) (`unstake`)

Code

```
// program/programs/tbb-staking/src/lib.rs:21-30
21 |     /// One-time: dev creates the pool. Authority = dev wallet.
22 |     pub fn initialize_pool(ctx: Context<InitializePool>) -> Result<()> {
23 |         let pool = &mut ctx.accounts.pool;
24 |         pool.authority = ctx.accounts.authority.key();
25 |         pool.mint = ctx.accounts.mint.key();
26 |         // @audit The mint is permanently accepted without rejecting a
27 |         //         non-null Token-2022 PermanentDelegate.
28 |         pool.treasury = ctx.accounts.treasury.key();
29 |         pool.total_staked = 0;
```

```

28 |         pool.total_stakes = 0;
29 |         pool.bump = ctx.bumps.pool;
30 |         Ok(())

// program/programs/tbb-staking/src/lib.rs:286-310
286 | #[derive(Accounts)]
287 | pub struct InitializePool<'info> {
288 |     #[account(mut)]
289 |     pub authority: Signer<'info>,
290 |     pub mint: InterfaceAccount<'info, Mint>,
    |     // @audit Token-2022 mints carrying a live PermanentDelegate satisfy
    |     // this account type; no extension or delegate validation exists.
291 |     #[account(
292 |         init,
293 |         payer = authority,
294 |         space = 8 + Pool::INIT_SPACE,
295 |         seeds = [b"pool"],
296 |         bump
297 |     )]
298 |     pub pool: Account<'info, Pool>,
299 |     #[account(
300 |         init,
301 |         payer = authority,
302 |         token::mint = mint,
303 |         token::authority = pool,
    |         // @audit Setting the base owner to `pool` does not exclude the
    |         // mint-level permanent delegate.
304 |         seeds = [b"treasury"],
305 |         bump
306 |     )]
307 |     pub treasury: InterfaceAccount<'info, TokenAccount>,
308 |     pub token_program: Interface<'info, TokenInterface>,
309 |     pub system_program: Program<'info, System>,
310 | }

// program/programs/tbb-staking/src/lib.rs:359-367
359 |     #[account(
360 |         init,
361 |         payer = staker,
362 |         token::mint = mint,
363 |         token::authority = stake_account,
    |         // @audit Every vault inherits the same mint-level delegate bypass,
    |         // despite `stake_account` being treated as its custodian.
364 |         seeds = [b"vault", stake_account.key().as_ref()],
365 |         bump
366 |     )]
367 |     pub vault: Box<InterfaceAccount<'info, TokenAccount>>,

// program/programs/tbb-staking/src/lib.rs:170-180
170 |     // 1) Return principal from the stake vault (authority = stake PDA).
171 |     transfer_tokens(
172 |         &ctx.accounts.token_program,
173 |         &ctx.accounts.vault,
174 |         &ctx.accounts.mint,
175 |         &ctx.accounts.staker_ata,
176 |         &ctx.accounts.stake_account.to_account_info(),
177 |         stake_acc.amount,
    |         // @audit This assumes no mint-level authority could have reduced
    |         // the vault below the recorded principal.
178 |         ctx.accounts.mint.decimals,
179 |         Some(&[stake_seeds]),
180 |     )?;

```

Mitigation Suggestion

During pool initialization, decode Token-2022 mint extensions and reject any mint with **PermanentDelegate** configured. Protocol custody must not be subject to a mint-level authority that can independently transfer or burn vault balances without the PDA owner.

Further, the initialization should ensure that at least 1 unit is always in circulation. This prevents an initializer from passing the init check, then closing the mint and reopening it with the **PermanentDelegate**. This can be done by ensuring the treasury is funded in the same IX and that it always has at least 1 unit of funding.

Remediation

Fixed in commit **50524af69cb7627d9598596b6ed1d84a596a2389**.

Summary

`initialize_pool` accepts either supported SPL Token mint without requiring the base `freeze_authority` to be disabled, even though the program treats PDA ownership of the treasury and stake vaults as sufficient to guarantee their future transferability; `stake` also counts a frozen treasury's nominal `amount` as available backing.

`InterfaceAccount<Mint>` accepts both the original SPL Token program and Token-2022, and a reachable sequence is for the intended authority to configure an otherwise valid freeze-enabled mint, allow an ordinary user to stake, and then have the legitimate freeze-authority signer freeze the funded vault and set the mint's freeze authority to `None`; the mature `unstake` thereafter fails on its first `TransferChecked`, rollback preserves the already-frozen vault and open stake, and the one-way authority revocation leaves neither the user nor either PDA able to thaw or move the principal.

Impact

Permanent principal freeze; the configured mint's legitimate freeze-authority signer can target one funded live vault and irreversibly revoke that authority, after which the affected ordinary staker cannot recover through retry or any existing instruction, while the trusted-role prerequisite caps the otherwise Critical end state at Medium.

Location

- [program/programs/tbb-staking/src/lib.rs:21-30](#) (`initialize_pool`)
- [program/programs/tbb-staking/src/lib.rs:91-100](#) (`stake`)
- [program/programs/tbb-staking/src/lib.rs:170-192](#) (`unstake`)
- [program/programs/tbb-staking/src/lib.rs:286-310](#) (`InitializePool`)
- [program/programs/tbb-staking/src/lib.rs:359-367](#) (`Stake`)

Code

```
// program/programs/tbb-staking/src/lib.rs:21-30
21 |     /// One-time: dev creates the pool. Authority = dev wallet.
22 |     pub fn initialize_pool(ctx: Context<InitializePool>) -> Result<()> {
23 |         let pool = &mut ctx.accounts.pool;
24 |         pool.authority = ctx.accounts.authority.key();
25 |         pool.mint = ctx.accounts.mint.key();
26 |         // @audit The mint is permanently accepted without requiring its
27 |         //         base freeze_authority to be None.
28 |         pool.treasury = ctx.accounts.treasury.key();
29 |         pool.total_staked = 0;
30 |         pool.total_stakes = 0;
31 |         pool.bump = ctx.bumps.pool;
32 |         Ok(())
33 |     }
34 |
35 |
36 |
37 |
38 |
39 |
40 |
41 |
42 |
43 |
44 |
45 |
46 |
47 |
48 |
49 |
50 |
51 |
52 |
53 |
54 |
55 |
56 |
57 |
58 |
59 |
60 |
61 |
62 |
63 |
64 |
65 |
66 |
67 |
68 |
69 |
70 |
71 |
72 |
73 |
74 |
75 |
76 |
77 |
78 |
79 |
80 |
81 |
82 |
83 |
84 |
85 |
86 |
87 |
88 |
89 |
90 |
91 |     // Treasury must already hold enough to honor this stake's interest,
92 |     // beyond what is already promised to earlier stakers.
93 |     let pool = &mut ctx.accounts.pool;
94 |     let available = ctx
95 |         .accounts
96 |         .treasury
97 |         .amount
98 |         // @audit Frozen accounts retain their nominal amount, so a
99 |         //         permanently frozen treasury is still treated as
100 |         //         spendable backing for new stakes.
101 |         .checked_sub(pool.total_promised_interest)
102 |         .ok_or(StakingError::TreasuryUnderfunded)?;
```

```

100|         require!(available >= interest, StakingError::TreasuryUnderfunded);

// program/programs/tbb-staking/src/lib.rs:170-192
170|         // 1) Return principal from the stake vault (authority = stake PDA).
171|         transfer_tokens(
172|             &ctx.accounts.token_program,
173|             &ctx.accounts.vault,
174|             &ctx.accounts.mint,
175|             &ctx.accounts.staker_ata,
176|             &ctx.accounts.stake_account.to_account_info(),
177|             stake_acc.amount,
178|             ctx.accounts.mint.decimals,
179|             Some(&[stake_seeds]),
180|         )?;
181|         // @audit TransferChecked rejects a frozen vault before moving any
182|         //         principal; PDA ownership does not bypass the frozen state.
183|
184|         // 2) Pay interest from treasury (authority = pool PDA).
185|         transfer_tokens(
186|             &ctx.accounts.token_program,
187|             &ctx.accounts.treasury,
188|             &ctx.accounts.mint,
189|             &ctx.accounts.staker_ata,
190|             &ctx.accounts.pool.to_account_info(),
191|             stake_acc.interest,
192|             ctx.accounts.mint.decimals,
193|             Some(&[pool_seeds]),
194|         )?;
195|         // @audit A frozen shared treasury also makes the entire unstake
196|         //         roll back, including the preceding principal transfer.

// program/programs/tbb-staking/src/lib.rs:286-310
286| #[derive(Accounts)]
287| pub struct InitializePool<'info> {
288|     #[account(mut)]
289|     pub authority: Signer<'info>,
290|     pub mint: InterfaceAccount<'info, Mint>,
291|     // @audit Both legacy SPL Token and Token-2022 mints with a live
292|     //         freeze authority satisfy this type; no authority check exists.
293|     #[account(
294|         init,
295|         payer = authority,
296|         space = 8 + Pool::INIT_SPACE,
297|         seeds = [b"pool"],
298|         bump
299|     )]
300|     pub pool: Account<'info, Pool>,
301|     #[account(
302|         init,
303|         payer = authority,
304|         token::mint = mint,
305|         token::authority = pool,
306|         // @audit Making the pool PDA the ordinary token-account owner does
307|         //         not restrict the mint's separate freeze authority.
308|         seeds = [b"treasury"],
309|         bump
310|     )]
311|     pub treasury: InterfaceAccount<'info, TokenAccount>,
312|     pub token_program: Interface<'info, TokenInterface>,
313|     pub system_program: Program<'info, System>,
314| }

// program/programs/tbb-staking/src/lib.rs:359-367
359|     #[account(
360|         init,
361|         payer = staker,
362|         token::mint = mint,
363|         token::authority = stake_account,
364|         // @audit The stake PDA is not exclusive custodian while the mint's
365|         //         freeze authority can independently freeze this vault.
366|         seeds = [b"vault", stake_account.key().as_ref()],
367|         bump

```

```
366|    )]  
367|    pub vault: Box<InterfaceAccount<'info, TokenAccount>>,
```

Mitigation Suggestion

During pool initialization, reject mints with a non-null base `freeze_authority`. This prevents a mint-level authority under either supported token program from freezing the treasury or stake vaults after users deposit.

Further, the initialization should ensure that at least 1 unit is always in circulation. This prevents an initializer from passing the init check, then closing the mint and reopening it with the freeze authority. This can be done by ensuring the treasury is funded in the same IX and that it always has at least 1 unit of funding.

Remediation

Fixed in `50524af69cb7627d9598596b6ed1d84a596a2389`.

ACC-I1

Hardcoded Interest table requires whole Program upgrade to change.

INFO

FIXED

Description

We found that the interest table is static and requires a whole program upgrade to change.

Location

<https://github.com/jwilliams-74w/tbb-staking/blob/152740e5b8c05fbea25ff9a995b9254db10cc8f1/program/programs/tbb-staking/src/lib.rs#L11-L18>

Relevant Code

```
/// lib.rs L11-L18
pub const TIERS: [(i64, u16); 5] = [
    (30 * 86_400, 500), // 1 month -> 5 APR
    (90 * 86_400, 800), // 3 months -> 8 APR
    (180 * 86_400, 1200), // 6 months -> 12 APR
    (365 * 86_400, 1800), // 12 months -> 18 APR
    (120, 1800), // DEMO: 2-minute lock (remove before mainnet)
];
```

Mitigation Suggestion

Create this table inside the pool and create an instruction that can modify the table. Ideally, include timeouts for cutover, such as epoch + 1, at which point the new table will take effect. Only the pool authority should be able to do so.

Remediation

Fixed in commit [50524af69cb7627d9598596b6ed1d84a596a2389](#).

Summary

`initialize_pool` authenticates no expected deployer and stores the first arbitrary signer as the authority of the single global pool PDA, while also accepting a caller-selected mint. A first caller can permanently occupy the only pool and treasury addresses and become the only signer accepted by the administrative instructions, with no authority-transfer, reset, or close path.

Impact

Initialization takeover; before one-time setup, any unprivileged signer can claim the pool, after which the legitimate operator cannot use the funding path or reclaim authority and the attacker can keep the treasury unfunded or remove every unreserved deposit until upgrade or redeployment.

Location

- [program/programs/tbb-staking/src/lib.rs:21-30](#) (`initialize_pool`)
- [program/programs/tbb-staking/src/lib.rs:286-309](#) (`InitializePool`)
- [program/programs/tbb-staking/src/lib.rs:312-338](#) (`FundTreasury`, `WithdrawSurplus`)

Code

```
// program/programs/tbb-staking/src/lib.rs:21-30
21 |     /// One-time: dev creates the pool. Authority = dev wallet.
22 |     pub fn initialize_pool(ctx: Context<InitializePool>) -> Result<()> {
23 |         let pool = &mut ctx.accounts.pool;
24 |         pool.authority = ctx.accounts.authority.key(); // @audit First arbitrary signer becomes the permanent authority.
25 |         pool.mint = ctx.accounts.mint.key();           // @audit The same caller chooses the global pool mint.
26 |         pool.treasury = ctx.accounts.treasury.key();
27 |         pool.total_staked = 0;
28 |         pool.total_stakes = 0;
29 |         pool.bump = ctx.bumps.pool;
30 |         Ok(())

// program/programs/tbb-staking/src/lib.rs:286-309
286 | #[derive(Accounts)]
287 | pub struct InitializePool<'info> {
288 |     #[account(mut)]
289 |     pub authority: Signer<'info>, // @audit No address or deployer-authority restriction.
290 |     pub mint: InterfaceAccount<'info, Mint>,
291 |     #[account(
292 |         init,
293 |         payer = authority,
294 |         space = 8 + Pool::INIT_SPACE,
295 |         seeds = [b"pool"], // @audit One global address; a first caller permanently occupies it.
296 |         bump
297 |     )]
298 |     pub pool: Account<'info, Pool>,
299 |     #[account(
300 |         init,
301 |         payer = authority,
302 |         token::mint = mint,
303 |         token::authority = pool,
304 |         seeds = [b"treasury"], // @audit One global treasury is occupied at the same time.
305 |         bump
306 |     )]
```

```

307 |     pub treasury: InterfaceAccount<'info, TokenAccount>,
308 |     pub token_program: Interface<'info, TokenInterface>,
309 |     pub system_program: Program<'info, System>,

// program/programs/tbb-staking/src/lib.rs:312-338
312 | #[derive(Accounts)]
313 | pub struct FundTreasury<'info> {
314 |     #[account(mut, address = pool.authority)]
315 |     pub authority: Signer<'info>, // @audit Only the first caller can use the administrative funding pa
th.
316 |     #[account(seeds = [b"pool"], bump = pool.bump, has_one = mint, has_one = treasury)]
317 |     pub pool: Account<'info, Pool>,
318 |     pub mint: InterfaceAccount<'info, Mint>,
319 |     #[account(mut)]
320 |     pub treasury: InterfaceAccount<'info, TokenAccount>,
321 |     #[account(mut, token::mint = mint, token::authority = authority)]
322 |     pub funder_ata: InterfaceAccount<'info, TokenAccount>,
323 |     pub token_program: Interface<'info, TokenInterface>,
324 | }
325 |
326 | #[derive(Accounts)]
327 | pub struct WithdrawSurplus<'info> {
328 |     #[account(mut, address = pool.authority)]
329 |     pub authority: Signer<'info>, // @audit The seized authority can withdraw all unreserved funding.
330 |     #[account(seeds = [b"pool"], bump = pool.bump, has_one = mint, has_one = treasury)]
331 |     pub pool: Account<'info, Pool>,
332 |     pub mint: InterfaceAccount<'info, Mint>,
333 |     #[account(mut)]
334 |     pub treasury: InterfaceAccount<'info, TokenAccount>,
335 |     #[account(mut, token::mint = mint, token::authority = authority)]
336 |     pub authority_ata: InterfaceAccount<'info, TokenAccount>,
337 |     pub token_program: Interface<'info, TokenInterface>,
338 | }

```

Remediation

Fixed in commit [50524af69cb7627d9598596b6ed1d84a596a2389](#).

Summary

The fixed-point interest calculation floors fractional rewards to whole token base units, but `stake` rejects only zero principal and never rejects a computed `interest` of zero. As a result, production-tier stakes of up to **243, 50, 16**, and **5** base units for tiers **0** through **3** respectively—and up to **1,459,999** base units for the demo tier—serve their complete lock while earning no return.

Impact

Zero-yield rounding; a staker who submits a positive below-threshold principal completes the full lock and receives zero interest, although the principal remains recoverable after maturity.

Location

- [programs/tbb-staking/src/lib.rs:81-89](#) (`stake`)
- [programs/tbb-staking/src/lib.rs:219-226](#) (`compute_interest`)

Code

```
// tbb-staking/programs/tbb-staking/src/lib.rs:81-89
81 | pub fn stake(ctx: Context<Stake>, amount: u64, tier: u8) -> Result<()> {
82 |     require!(amount > 0, StakingError::ZeroAmount);
83 |     // @audit: Any positive base-unit amount is accepted; there is no minimum
84 |     // amount based on the selected tier's reward precision.
85 |     require!((tier as usize) < TIERS.len(), StakingError::InvalidTier);
86 |
87 |     let (lock_seconds, apr_bps) = TIERS[tier as usize];
88 |     let now = Clock::get()?.unix_timestamp;
89 |
90 |     // Interest owed at maturity, computed up front (fixed-term product).
91 |     let interest = compute_interest(amount, apr_bps, lock_seconds)?;
92 |     // @audit: A zero result is accepted and the principal is still locked.
93 |
94 |
95 |
96 |
97 |
98 |
99 |
100|
101|
102|
103|
104|
105|
106|
107|
108|
109|
110|
111|
112|
113|
114|
115|
116|
117|
118|
119|
120|
121|
122|
123|
124|
125|
126|
127|
128|
129|
130|
131|
132|
133|
134|
135|
136|
137|
138|
139|
140|
141|
142|
143|
144|
145|
146|
147|
148|
149|
150|
151|
152|
153|
154|
155|
156|
157|
158|
159|
160|
161|
162|
163|
164|
165|
166|
167|
168|
169|
170|
171|
172|
173|
174|
175|
176|
177|
178|
179|
180|
181|
182|
183|
184|
185|
186|
187|
188|
189|
190|
191|
192|
193|
194|
195|
196|
197|
198|
199|
200|
201|
202|
203|
204|
205|
206|
207|
208|
209|
210|
211|
212|
213|
214|
215|
216|
217|
218|
219 | fn compute_interest(amount: u64, apr_bps: u16, lock_seconds: i64) -> Result<u64> {
220 |     let interest = (amount as u128)
221 |         .checked_mul(apr_bps as u128)
222 |         .and_then(|v| v.checked_mul(lock_seconds as u128))
223 |         .and_then(|v| v.checked_div(10_000))
224 |         .and_then(|v| v.checked_div(SECONDS_PER_YEAR))
225 |         // @audit: Integer division floors sub-base-unit interest to zero.
226 |         .ok_or(StakingError::MathOverflow)?;
227 |     u64::try_from(interest).map_err(|_| StakingError::MathOverflow.into())
228 | }
```

Mitigation Suggestion

Require interest to be greater than 0

Remediation

Fixed in [50524af69cb7627d9598596b6ed1d84a596a2389](#).

APPENDIX

Vulnerability Classification

We rate our issues according to the following scale. Informational issues are reported informally to the developers and are not included in this report.

Severity	Description
Critical	Vulnerabilities that can be easily exploited and result in loss of user funds, or directly violate the protocol's integrity. Immediate action is required.
High	Vulnerabilities that can lead to loss of user funds under non-trivial preconditions, loss of fees, or permanent denial of service that requires a program upgrade. These issues require attention and should be resolved in the short term.
Medium	Vulnerabilities that may be more difficult to exploit but could still lead to some compromise of the system's functionality. For example, partial denial of service attacks, or such attacks that do not require a program upgrade to resolve, but may require manual intervention. These issues should be addressed as part of the normal development cycle.
Low	Vulnerabilities that have a minimal impact on the system's operations and can be fixed over time. These issues may include inconsistencies in state, or require such high capital investments that they are not exploitable profitably.
Informational	Findings that do not pose an immediate risk but could affect the system's efficiency, maintainability, or best practices.

Audit Methodology

Accretion is a boutique security auditor specializing in Solana's ecosystem. We employ a customized approach for each client, strategically allocating our resources to maximize code review effectiveness. Our auditors dedicate substantial time to developing a comprehensive understanding of each program under review, examining design decisions, expected and edge-case behaviors, invariants, optimizations, and data structures, while meticulously verifying mathematical correctness—all within the context of the developers' intentions.

Our audit scope extends beyond on-chain components to include associated infrastructure, such as user interfaces and supporting systems. Every audit encompasses both a holistic protocol design review and detailed line-by-line code analysis.

During our assessment, we focus on identifying:

- Solana-specific vulnerabilities
- Access control issues
- Arithmetic errors and precision loss
- Race conditions and MEV opportunities
- Logic errors and edge cases
- Performance optimization opportunities
- Invariant violations
- Account confusion vulnerabilities
- Authority check omissions
- Token22 implementation risks and SPL-related pitfalls
- Deviations from best practices

Our approach transcends conventional vulnerability classifications. We continuously conduct ecosystem-wide security research to identify and mitigate emerging threat vectors, ensuring our audits remain at the forefront of Solana security practices.